

AFRILANG Stdlib

std/c/cryptxor

Français. Bibliothèque AFRILANG 'std/c/cryptxor' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : xorBytes, xorHex, repeatKey, xorList, parity, rollingXor, selfXor.

```
'import "std/c/cryptxor"' · 'use cryptxor'
```

- 'xorBytes(a text, b text) → text' - 'xorHex(hex text, key text) → text'
- 'repeatKey(data text, key text) → text' - 'xorList(a list number, b list number) → list number' - 'parity(s text) → number' - 'rollingXor(data list number, key number) → list number' - 'selfXor(s text) → text' **En-**

glish. AFRILANG library 'std/c/cryptxor' (tier complex, category complex). Exports 7 function(s): xorBytes, xorHex, repeatKey, xorList, parity, rollingXor, selfXor.

```
'import "std/c/cryptxor"' · 'use cryptxor'
```

- 'xorBytes(a text, b text) → text' - 'xorHex(hex text, key text) → text'
- 'repeatKey(data text, key text) → text' - 'xorList(a list number, b list number) → list number' - 'parity(s text) → number' - 'rollingXor(data list number, key number) → list number' - 'selfXor(s text) → text'

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/cryptxor' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : xorBytes, xorHex, repeatKey, xorList, parity, rollingXor, selfXor.

```
'import "std/c/cryptxor"' · 'use cryptxor'
```

```
- `xorBytes(a text, b text) → text`
- `xorHex(hex text, key text) → text`
- `repeatKey(data text, key text) → text`
- `xorList(a list number, b list number) → list number`
- `parity(s text) → number`
- `rollingXor(data list number, key number) → list number`
- `selfXor(s text) → text`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/cryptxor"
use cryptxor
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- xorBytes(a text, b text) → text•
xorHex(hex text, key text) → text•
repeatKey(data text, key text) → text•
xorList(a list number, b list number) → list•
parity(s text) → number•
rollingXor(data list number, key number) → list•
selfXor(s text) → text

4. Exemples d'utilisation

```
import "std/c/cryptxor"
use cryptxor
```

```
create result = xorBytes(/* args */)
say result
```

```
create result = xorHex(/* args */)
say result
```

```
create result = repeatKey(/* args */)
say result
```

```
create result = xorList(/* args */)
say result
```

```
create result = parity(/* args */)
say result
```

```
create result = rollingXor(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/cryptxor"`` en tête de fichier.
- Lancez ``afri-lang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module cryptxor

```
function xorBytes(a text, b text) returns text
end
```

```
function xorHex(hex text, key text) returns text
end
```

```
function repeatKey(data text, key text) returns text
end
```

```
function xorList(a list number, b list number) returns list number
end
```

```
function parity(s text) returns number
end
```

```
function rollingXor(data list number, key number) returns list number
end
```

```
function selfXor(s text) returns text
end
end
```

English

1. Overview

AFRILANG library 'std/c/cryptxor' (tier complex, category complex). Exports 7 function(s): xorBytes, xorHex, repeatKey, xorList, parity, rollingXor, selfXor.

'import "std/c/cryptxor"' · 'use cryptxor'

```
- `xorBytes(a text, b text) → text`
- `xorHex(hex text, key text) → text`
- `repeatKey(data text, key text) → text`
- `xorList(a list number, b list number) → list number`
- `parity(s text) → number`
- `rollingXor(data list number, key number) → list number`
- `selfXor(s text) → text`
```

2. Installation & import

Add the module to your program:

```
import "std/c/cryptxor"
use cryptxor
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- xorBytes(a text, b text) → text•
xorHex(hex text, key text) → text•
repeatKey(data text, key text) → text•
xorList(a list number, b list number) → list•
parity(s text) → number•
rollingXor(data list number, key number) → list•
selfXor(s text) → text

4. Usage examples

```
import "std/c/cryptxor"
use cryptxor
```

```
create result = xorBytes(/* args */)
say result
```

```
create result = xorHex(/* args */)
say result
```

```
create result = repeatKey(/* args */)
say result
```

```
create result = xorList(/* args */)
say result
```

```
create result = parity(/* args */)
say result
```

```
create result = rollingXor(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/cryptxor"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module cryptxor

```
function xorBytes(a text, b text) returns text
end
```

```
function xorHex(hex text, key text) returns text
end
```

```
function repeatKey(data text, key text) returns text
end
```

```
function xorList(a list number, b list number) returns list number
end
```

```
function parity(s text) returns number
end
```

```
function rollingXor(data list number, key number) returns list number
end
```

```
function selfXor(s text) returns text
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/cryptxor"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/cryptxor/>

Source (extrait)

```
module cryptxor

function xorBytes(a text, b text) returns text
end

function xorHex(hex text, key text) returns text
```

```
end

function repeatKey(data text, key text) returns text
end

function xorList(a list number, b list number) returns list number
end

function parity(s text) returns number
end

function rollingXor(data list number, key number) returns list number
end

function selfXor(s text) returns text
end
end
```